An Empirical Study of Software Sanitization Locality

Nathaniel Christian Boland
Wright State University
boland.5@wright.edu

Junjie Zhang
Wright State University
junjie.zhang@wright.edu

Daniel Chong
Wright State University
chong.8@wright.edu

Rui Dai
University of Cincinnati
rui.dai@uc.edu

***Abstract*—This work introduces the concept of software sanitization locality and conducts empirical measurements. We define software sanitization locality as the property wherein the sanitization operation, if present, remains proximate to its protected API. To quantify this property, we have introduced a range of metrics to illustrate the distance between a sanitization operation and its protected API from various perspectives, including both the abstract syntax tree level and the binary level. In an effort to validate the concept of sanitization locality, we have also gathered and labeled a dataset of programs containing security patches to conduct empirical measurements. This dataset encompasses a diverse array of 16 typical vulnerabilities sourced from the Linux kernel codebase. The findings conclusively illustrate that the analyzed samples do exhibit the hypothesized sanitization locality.**

Keywords*—*Software security, vulnerabilities

I. Introduction

Detecting software vulnerabilities is imperative to secure cyberspace. However, the expansive scale and high complexity of modern software present a huge challenge to this endeavor. For example, it becomes impossible to perform manual code auditing for large software. Despite numerous proposed automated software vulnerability detection methods, their effectiveness and efficiency are still impeded by the compounded challenges of size and complexity. For example, symbolic execution [1], a fundamental method for software vulnerability detection, is inherently hindered by the path explosion challenge [2] in programs with many branches. We aim to tackle this challenge from a distinct angle by addressing a fundamental yet intuitive question: "Is analyzing the entirety of a program necessary for effective vulnerability detection?"

Our work starts with a common observation: numerous vulnerabilities stem from the misuse of sensitive APIs lacking proper sanitization of inputs. Salient examples include memory overflows, integer overflows, format string vulnerabilities, SQL injections, and cross-site scripting vulnerabilities. We have proposed a hypothesis, namely *software sanitization locality*, suggesting that the sanitization operation guarding a sensitive API, if present, remains closely to that API. This hypothesis, if confirmed, bears strong practical implications, suggesting that effective vulnerability detection can be achieved by scrutinizing a confined segment of the code base.

Despite its significance, validating this hypothesis encounters huge hurdles. Primarily, there are currently no defined metrics to measure software sanitization locality. Secondly, labeled data for validation is currently unavailable. Ideally, one would need to conduct a comprehensive scan of a program, identifying sensitive APIs, evaluating their security implications, and ultimately pinpointing any associated sanitization operations. However, it is a major challenge to assess whether a sensitive API genuinely poses vulnerability risks. In addition, the process of scanning a large set of programs, identifying sensitive APIs, and validating them is prohibitively resource-intensive.

To tackle these challenges, we've introduced a series of metrics to quantify the distance between a sensitive API and its corresponding sanitization operations. These metrics quantify the proposed locality across various program representations, including abstract syntax trees (AST) and executable binary. To enhance the precision and efficiency of data labeling, we have used code bases containing security patches. Such repositories provide focused areas for identifying both sensitive APIs and their potential sanitization operations. It's important to note that while patched code bases, with added sanitization operations, offer practical feasibility for data collection, they only approximate code bases where sanitization operations are inherently embedded. Given that our metrics profile programs across different representation levels, collected code bases must be compilable. Through this project, we have contributed a labeled dataset, a set of metrics, and experimental results. These contributions together build an empirical foundation for investigating software sanitization locality and hold significant potential for efficiently detecting vulnerabilities through targeted examination of specific segments within the code base.

II. Related Work

Methods of software vulnerability detection mainly leverage program analysis techniques, such as taint analysis [3], [4], symbolic execution [1], [5], and fuzzing [6], [7]. Many of these techniques encounter scalability challenges as the size, complexity, or both of a program increases. Significant research efforts have been invested to improve their efficiency so that they can be applied to real-world applications whose size and complexity are typically both large. For example, J. Dahse et. al. [8] have proposed to perform taint analysis based on summaries of functions instead of carrying out inter-procedural analysis. Symbolic execution is inherently limited by the path explosion challenge [2]. Efforts have been made to mitigate this chal-

Aspects	Metrics
Path Size	# of all AST nodes
Branches	# of if statements # of ternary statements # of switch statements # of case statements
Loops	# of while statements # of do-while statements # of for statements
Breaks	# of break statements # of goto statements
Functions	# of function-call statements # of function-declaration statements # of return statements

TABLE I: The Proposed 13 Metrics to Measure Distance From 5 Aspects of an AST

lenge by concolic execution [9] or heuristically reducing constraints [10]. Fuzzing typically features a huge overhead in analysis time, where strategies such as directed fuzzing [7] and smart seeding [11] have been used for the expedition. Nevertheless, all of them focus on improving the analysis methods rather than exploring the possibility of intelligently reducing the size of the program under analysis.

To the best of our knowledge, two projects including *UChecker* [12] and *UFuzzer* [13] tackled a similar concept of software sanitization vulnerability. Specifically, these two systems leverage heuristics to identify a small portion of code that is likely to be relevant to the unrestricted file upload vulnerability [14] in PHP. Both systems have accomplished high detection accuracy and high efficiency by analyzing the identified small portion of the code. Nevertheless, they focus on a single type of vulnerability, i.e., the unrestricted file upload, and they are designed for server-side web programs in PHP. Neither of these systems delves into the extent to which software sanitization locality applies to other classes of vulnerabilities.

A few datasets of vulnerable source code, such as BigVul [15], Draper VDISC [16], Juliet 1.3 [17], are publicly available. Unfortunately, these datasets themselves are applicable for our study since their vulnerabilities lack, rather than include, sanitization code that protects a sensitive API. Wang et. al. [18] also proposed a method that leverages deep learning to automatically identify source code with security patches from GitHub. Nevertheless, this work cannot infer the specific type or the location of the vulnerability protected by a security patch. In addition, compiling these code bases is very challenging in practice considering the high diversity and complexity of their compilation process such as the code and environment dependency. To tackle this challenge, we have leveraged patched programs sourced from Linux kernels. This choice is grounded in two practical considerations. Firstly, it allows us to trace a security patch back to its corresponding vulnerability. Secondly, these patched programs share similar and stable code bases, i.e., as the Linux kernels, making it practically feasible to address their dependency requirements.

III. Metrics To Quantify Locality

We propose a series of metrics designed to measure software sanitization locality across both the abstract syntax tree and binary levels.

Listing 1 Locality Analysis Example.

```
1   int vulnerableExample(int* array, int index, int size)
2   {
3       if(size < 10) // Patch
4           return -1;
5
6       while (invalidIndex(index, size)){
7           index -= 10;
8       }
9
10      return modifyArray(array, index);
11  }
12
13  bool invalidIndex(int index, int size)
14  {
15      return index >= size;
16  }
17
18  int modifyArray(int* array, int index)
19  {
20      int returnValue = array[index]; // Vulnerability
21
22      array[index] = 0;
23
24      return returnValue;
25  }
26
```

A. AST Level Metrics

It typically takes trivial efforts to transfer a program without grammar error from its source code to the AST. Compared to source code, an AST ideally represents the syntax of a program while simultaneously excluding superfluous data such as comments and empty lines. We measure a metric by traversing an AST from the sanitization code towards the protected API. Nevertheless, such tree traversal is practically complicated by the high complexity of the AST. For example, one execution path may never lead to the invocation of the sensitive API. In order to address this challenge, we use the following rules to efficiently traverse the AST and determine a single path.

1) Rule 1: For an if-else statement, if the "true" branch does not contain the sensitive API, we will always traverse the "false" branch if existing.
2) Rule 2: The body of a loop statement will only be traversed once.
3) Rule 3: We will not traverse into the body of an invoked function unless the sensitive API resides in the body of the function or the callees of this invoked function.

We follow these rules to traverse an AST from the sanitization code to its protected API, resulting in a path of AST nodes. Along this path, we next define 13 metrics to quantify the distance between the sanitization code and its protected API by counting specific types of AST nodes. These 13 metrics characterize the distance from four different aspects including i) the total number of AST nodes, ii) the number of branches, iii) the number of loops, and iv) the number of breaks, and iv) the number of functions. Table I presents these 13 metrics.

In order to illustrate these metrics, we use a hypothetical example shown in Listing 1. Without the patch on lines 3 and 4, an out-of-bounds array access could occur on line

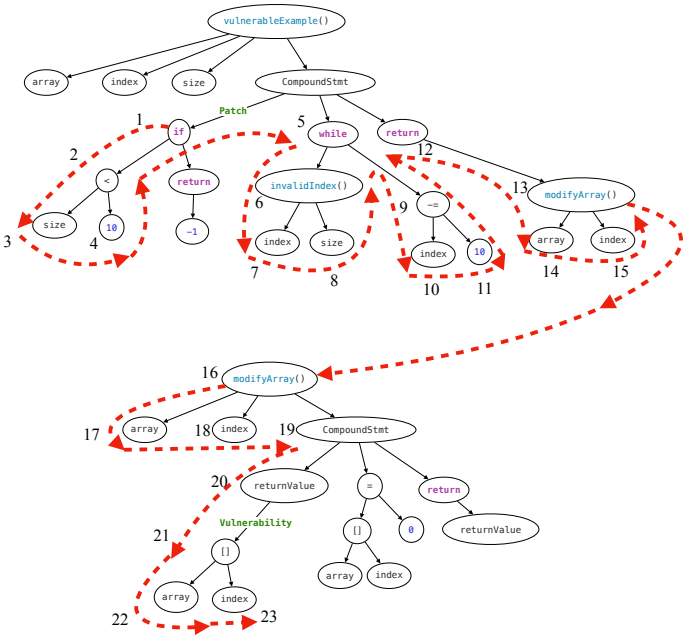


Fig. 1: ASTs from Code in Listing 1 and The Traversed Path

Metrics	Values
# of all AST nodes	23
# of if statements	1
# of ternary statements	0
# of switch statements	0
# of case statements	0
# of while statements	1
# of do-while statements	0
# of for statements	0
# of break statements	0
# of goto statements	0
# of function-call statements	2
# of function-declaration statements	1
# of return statements	1

TABLE II: Distance Quantified By 13 AST Metrics for ASTs from Code in Listing 1

20. The abstract syntax trees parsed from this example are visualized in Figure 1.

We will need to follow our AST traversing rules to identify the path. Specifically, the first step that would be performed for gathering the distance metrics for this sample would be deciding what line is the patch line and what line is the vulnerable line. The patch line for this example is line 3. Line 3 is chosen for the patch line because it is the line where the variable `size` is checked if it is too small. The vulnerable line is line 20. Next, the path for analysis would need to be found. The if-statement body on line 4 is not entered because if it were, the function would exit before reaching the previously vulnerable line. The while-loop is entered because of Rule 2. The function body starting on line 20 is counted in the path of analysis due to Rule 3. The identified path is presented using a dotted red line in Figure 1. We next measure this path using 13 metrics for AST, where their values are presented in Table II.

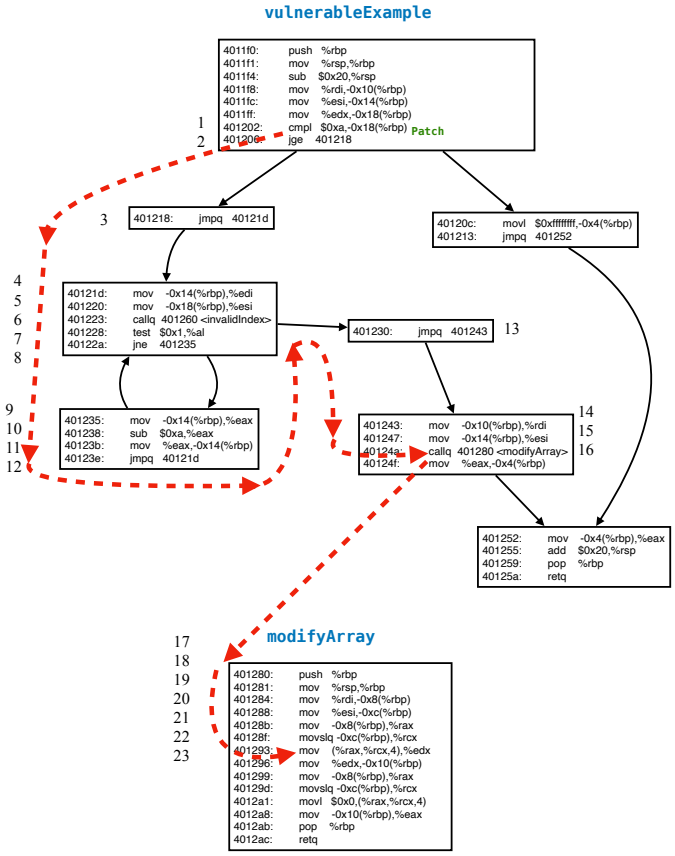


Fig. 2: CFG of The Binary Compiled from Code in Listing 1

B. Binary Level Metrics

The locality analysis at the binary level has significant implications since it can directly binary-level analysis such as taint analysis, symbolic execution, and emulation. We perform binary-level analysis by disassembling a binary into its assembly instructions and organizing them into a set of control flow graphs (CFG).

Specifically, we compile a `c` program into an x86 binary using `Clang` and then disassemble the binary into the assembly using `Ghidra`. The successful compilation requires all functions and data types (or at least functions and data types satisfying the signatures in the source code) in the source code to be provided.

For consistency, we use the same rules in AST-level analysis to identify a path from the sanitization code to the protected API by traversing the CFG. Along each path, we define five metrics to quantify the distance between the sanitization code and the protected API from three aspects including the total number of instructions, branches, and functions. Table III presents each metric and its corresponding category.

Figure 2 presents the CFG of the binary compiled from the example code in Listing 1. Line 0x401202 in Figure 2 is the beginning of the sanitization code (i.e., the patch). Line 0x401293 corresponds to out-of-bounds array access vulnerability (i.e., for Line 20 of Listing 1). The path is indicated using a red dotted line. Values of these metrics for the binary in Figure 2 are presented in Table IV.

Aspects	Metrics
Path Size	# of all instructions
Branches	# of jumps # of conditional jumps
Functions	# of function calls # of function declarations

TABLE III: The Proposed 5 Metrics to Measure Distance From 3 Aspects of a Binary

Metrics	Values
# of Instructions	23
# of Jumps	5
# of Conditional Jumps	2
# of Function Calls	2
# of Function Declarations	1

TABLE IV: Distance Quantified By 5 Binary Metrics for the Binary from Code in Listing 1

IV. Data Collection and Labeling

We leveraged the Big-Vul [15] dataset to bootstrap our data collection and labeling. Big-Vul, which has 3,754 vulnerable samples, was built by crawling the Common Vulnerabilities and Exposures (CVE) database [19] and connecting the CVEs with open-source GitHub projects. When links to patched code on GitHub are available, we can use GitHub's source control feature to retrieve past versions of patched code. By manually analyzing patch comments, the patched, and the unpatched code, we can successfully discover the exact location of the vulnerability being patched.

While retrieving the patched code bases from Big-Vul and GitHub is manageable, it is a daunting task to compile them into abstract syntax trees and binaries. Specifically, the diversity of code dependency is overwhelming, making it practically infeasible to individually address the dependency issue for each code base. Hence, we have used patched code bases from Linux kernels, which apparently contribute to a large number of vulnerable samples in the Big-Vul dataset. Unfortunately, Linux kernels themselves involve a wide variety of versions. Due to the complexity and large size of the Linux kernel, it was infeasible to compile the entire Linux kernel for each version.

Our solution is to compile a single version of the Linux kernel (5.13-rc7). Next, we insert into the compiled kernel each patched code sample and recompile it incrementally. It is worth noting that different versions of the Linux kernel can have substantial differences, which sometimes require us to add our own code to make the inserted code compile. These additions typically include structs, marcos, and functions. Nevertheless, this process drastically reduces the resource demands of compiling the Linux kernel.

By using this process, we have been able to successfully collect 73 patched code bases that cover a variety of vulnerabilities for the C-language. 57 of them have been successfully compiled into binaries in x86 using `Clang`. These samples include a wide variety of vulnerability types, which are shown in Table V.

V. Results

We measure the distance between the sanitization code and the protected API using proposed metrics. Specifically,

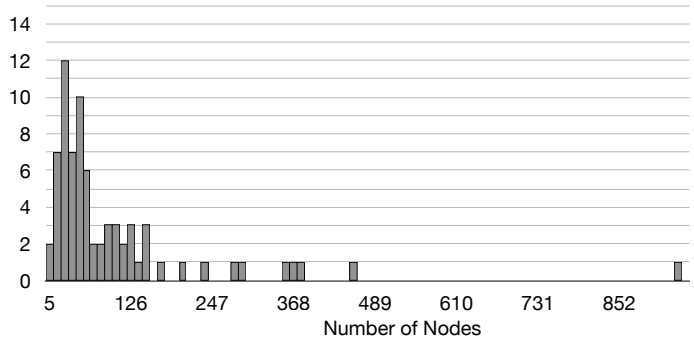


Fig. 3: A histogram showing the distance measured by the AST-based Number of Nodes metric.

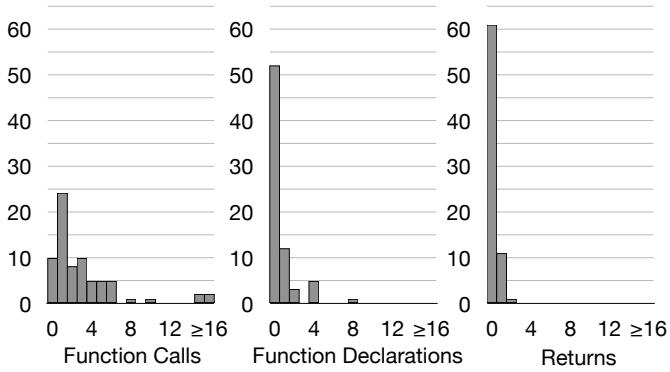


Fig. 4: Histograms showing distances measured by AST-based functions-related metrics.

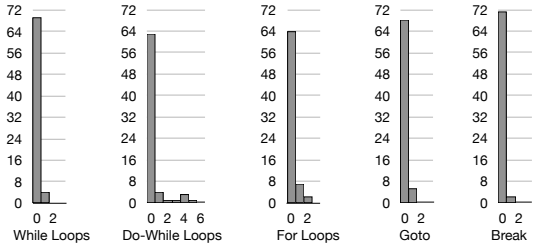


Fig. 5: Histograms showing distances measured by AST-based loops- and breaks-related metrics.

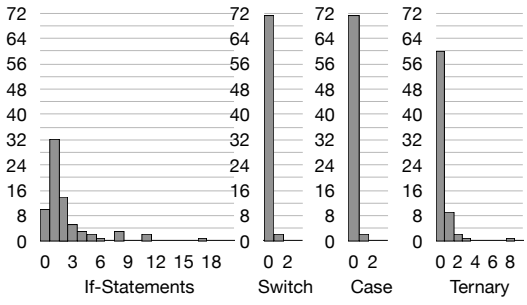


Fig. 6: Histograms showing distances measured by AST-based branches-related metrics.

Vulnerabilities	# of ASTs	# of Binaries
Buffer Overflow	8	8
Divide By Zero	3	1
Flipped Operands	1	0
Heap-based Buffer Overflow	3	2
Heap-based Memory Corruption	1	1
Improper Checking	6	5
Improper Initialization	13	11
Infinite Loop	1	1
Integer Overflow	7	3
Integer Signedness	1	0
Integer Underflow	2	1
Invalid Pointer Dereference	2	2
Null Pointer Dereference	17	15
Out of Bounds Array Access	3	3
Stack-based Buffer Overflow	3	3
Use After Free	2	1
Total	**73**	**57**

TABLE V: ASTs and Binaries of Patched Vulnerable Samples

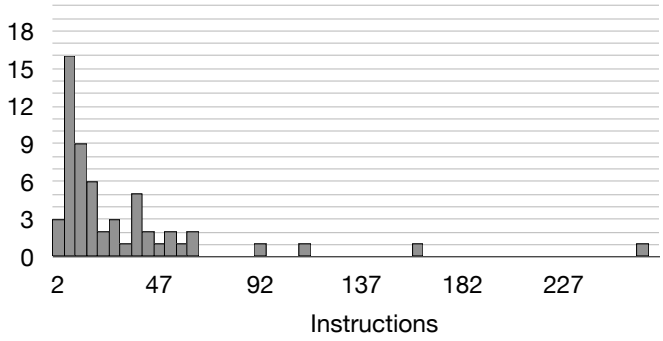


Fig. 7: A histogram showing the distance measured by the binary-based Number of Instructions metric.

histograms of distances measured by 13 AST-based metrics are presented in Figure 3, Figure 4, Figure 5, and Figure 6. All these distributions have demonstrated that the sanitization code stays very close to the protected API. For example, the vast majority of studied samples have the value of 0 measured by the numbers of function declarations, returns, while loops, do-while loops, for loops, gotos, breaks, switches, cases, and ternaries. The vast majority of samples have less than a small number of 6 invoked functions between the sanitization code and the protected API. Similarly, there are less than 6 if-statements observed for most samples. The distance measured by the number of nodes is slightly larger than those measured by other metrics. This is anticipated since this metric considers all types of AST nodes. Yet, the value of this metric tends to be small (e.g., less than 500) considering the fact that an AST of an application program typically has a large number of nodes. Histograms of distances measured by 5 binary-based metrics are presented in Figure 7 and Figure 8. Similar to AST-based metrics, all binary-based metrics tend to have small values. This demonstrates that the sanitization code stays very close to the protected API in binaries. For example, the vast majority of studied samples have the value of 0 measured by the number of function declarations. Most samples have the number of jumps and the number of conditional jumps smaller than 9. Meanwhile, there are typically less than 5 function calls between the sanitization code and the protected API. Finally, the distance measured

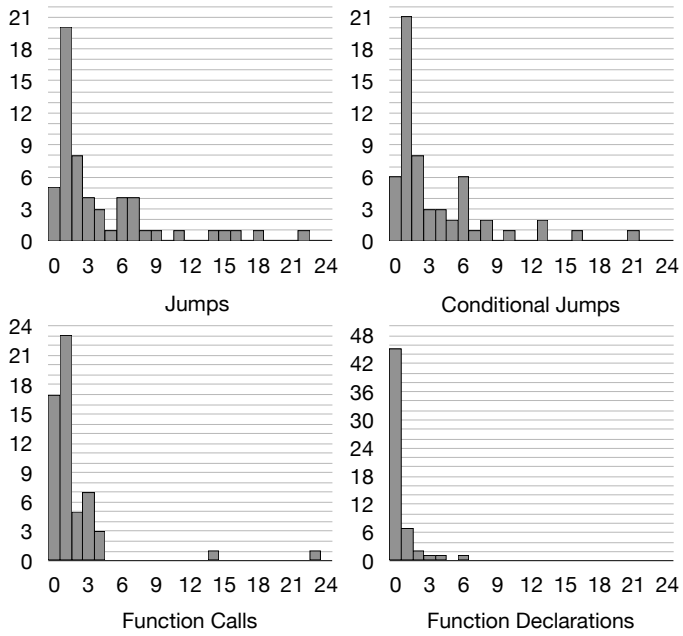


Fig. 8: Histograms showing distances measured by binary-based branches- and functions-related metrics.

by the number of instructions is also small, indicating a small number of instructions (i.e., less than 50).

VI. Insights

The validated *software sanitization locality* hypothesis has strong practical implications. Specifically, it suggests that effective vulnerability detection can be achieved by analyzing a confined segment of code base. This holds the promise of substantially reducing the computational overhead for intensive program analysis methods like concolic and symbolic execution.

The full potential of the software sanitization locality hypothesis can be realized through automated inference of the code base segment that is likely to contain sanitization operations given a sensitive API. Our proposed distance metrics can function as features for training machine learning models to statistically infer such associations. Specifically, a statistical model could potentially trace back from a sensitive API and pinpoint the instruction most likely to serve as the boundary of the code segment. However, collecting a sizable dataset of program samples with labeled information, including sensitive APIs and their associated sanitization operations, is an extremely challenging task. Our approach employs a "post-development" labeling strategy, which proves labor-intensive, tedious, and error-prone. Hence, we advocate for integrating the labeling task into the software development process. This move will result in precisely-labeled program samples that can significantly enhance software vulnerability detection and streamline the training of relevant statistical models for automated program analysis.

VII. Conclusion

Our study has empirically validated the *software sanitization locality* hypothesis, confirming that the sanitization operation, if present, tends to be in close proximity to the protected sensitive API. This validated conclusion carries significant practical implications, suggesting that effective vulnerability detection can be achieved through analysis of a localized portion of the code base.

References

[1] J. C. King, "Symbolic execution and program testing," *Communications of the ACM*, vol. 19, no. 7, pp. 385–394, 1976.

[2] N. Stephens, J. Grosen, C. Salls, A. Dutcher, R. Wang, J. Corbetta, Y. Shoshitaishvili, C. Kruegel, and G. Vigna, "Driller: Augmenting fuzzing through selective symbolic execution." in *NDSS*, vol. 16, no. 2016, 2016, pp. 1–16.

[3] J. Newsome and D. X. Song, "Dynamic taint analysis for automatic detection, analysis, and signature generation of exploits on commodity software." in *NDSS*, vol. 5. Citeseer, 2005, pp. 3–4.

[4] A. Sabelfeld and A. C. Myers, "Language-based information-flow security," *IEEE Journal on selected areas in communications*, vol. 21, no. 1, pp. 5–19, 2003.

[5] R. Baldoni, E. Coppa, D. C. D'elia, C. Demetrescu, and I. Finocchi, "A survey of symbolic execution techniques," *ACM Computing Surveys (CSUR)*, vol. 51, no. 3, pp. 1–39, 2018.

[6] J. Li, B. Zhao, and C. Zhang, "Fuzzing: a survey," *Cybersecurity*, vol. 1, no. 1, pp. 1–13, 2018.

[7] M. Böhme, V.-T. Pham, M.-D. Nguyen, and A. Roychoudhury, "Directed greybox fuzzing," in *Proceedings of the 2017 ACM SIGSAC conference on computer and communications security*, 2017, pp. 2329–2344.

[8] J. Dahse and T. Holz, "Simulation of built-in php features for precise static code analysis." in *NDSS*, vol. 14, 2014, pp. 23–26.

[9] I. Yun, S. Lee, M. Xu, Y. Jang, and T. Kim, "{QSYM}: A practical concolic execution engine tailored for hybrid fuzzing," in *27th USENIX Security Symposium (USENIX Security 18)*, 2018, pp. 745–761.

[10] X. Mi, S. Rawat, C. Giuffrida, and H. Bos, "Leansym: Efficient hybrid fuzzing through conservative constraint debloating," in *Proceedings of the 24th International Symposium on Research in Attacks, Intrusions and Defenses*, 2021, pp. 62–77.

[11] A. Rebert, S. K. Cha, T. Avgerinos, J. Foote, D. Warren, G. Grieco, and D. Brumley, "Optimizing seed selection for fuzzing," in *23rd USENIX Security Symposium (USENIX Security 14)*, 2014, pp. 861–875.

[12] J. Huang, Y. Li, J. Zhang, and R. Dai, "Uchecker: Automatically detecting php-based unrestricted file upload vulnerabilities," in *2019 49th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*. IEEE, 2019, pp. 581–592.

[13] J. Huang, J. Zhang, J. Liu, C. Li, and R. Dai, "Ufuzzer: Lightweight detection of php-based unrestricted file upload vulnerabilities via static-fuzzing co-analysis," in *Proceedings of the 24th International Symposium on Research in Attacks, Intrusions and Defenses*, 2021, pp. 78–90.

[14] T. Lee, S. Wi, S. Lee, and S. Son, "Fuse: Finding file upload bugs via penetration testing." in *NDSS*, 2020.

[15] J. Fan, Y. Li, S. Wang, and T. N. Nguyen, "A c/c++ code vulnerability dataset with code changes and cve summaries," in *Proceedings of the 17th International Conference on Mining Software Repositories*, 2020, pp. 508–512.

[16] R. Russell, L. Kim, L. Hamilton, T. Lazovich, J. Harer, O. Ozdemir, P. Ellingwood, and M. McConley, "Automated vulnerability detection in source code using deep representation learning," in *2018 17th IEEE international conference on machine learning and applications (ICMLA)*. IEEE, 2018, pp. 757–762.

[17] P. E. Black and P. E. Black, *Juliet 1.3 Test Suite: Changes From 1.2*. US Department of Commerce, National Institute of Standards and Technology, 2018.

[18] S. Wang, X. Wang, K. Sun, S. Jajodia, H. Wang, and Q. Li, "Graphspd: Graph-based security patch detection with enriched code semantics," in *2023 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2023, pp. 2409–2426.

[19] P. Mell and T. Grance, "Use of the common vulnerabilities and exposures (cve) vulnerability naming scheme," *NIST Special Publication*, vol. 800, p. 51, 2002.